

An Empirical Comparison of Scheduling Policies in a Distributed Task Queue

Latency, deadline compliance, fairness and starvation under controlled load

Anish Choudhury

September 2026

Contents

Abstract	1
1. Introduction	2
2. System design	2
3. Methodology	4
4. Results	5
5. Threats to validity	9
6. Conclusions	11
Appendix A: Reproducing these results	12
Appendix B: Experiment inventory	12

Abstract

Most distributed task queues hard-code a single queue discipline, almost always arrival order. This report treats the discipline as an experimental variable. A task queue was built in Go on Redis Streams in which the scheduling policy is a pluggable component chosen by configuration, and four policies — FIFO, strict priority, earliest deadline first (EDF) and weighted fair queuing (WFQ) — were compared across four synthetic workloads, seven levels of offered load, and two models of task execution.

The study comprises **1,264 experiments over 4.48 million tasks**. Three results are reported. First, scheduling policy is decisive only in a narrow band around service capacity: the spread between the best and worst policy’s 99th-percentile latency is 0.1% at half capacity, peaks at 93% at capacity, and falls to 1.1% at three times capacity. Second, centralising the scheduling decision costs approximately 1.5% of throughput, while *using* that decision costs 6–10% and returns improvements of one to two orders of magnitude in the metric each policy targets. Third, and least expected, the common practice of simulating task execution with sleeps is not a neutral simplification: it systematically flatters deadline-aware scheduling, because EDF’s advantage depends on declared task durations that real execution invalidates. WFQ is unaffected by the same error, and the asymmetry has a clean explanation.

Three quantitative claims made earlier in this work were subsequently found to be wrong and are corrected here. That process is reported rather than concealed, because the corrections are among the more instructive outcomes.

1. Introduction

1.1 Research question

How does the choice of scheduling policy affect latency, throughput, deadline compliance, fairness and starvation in a distributed task queue under different workload distributions and levels of offered load?

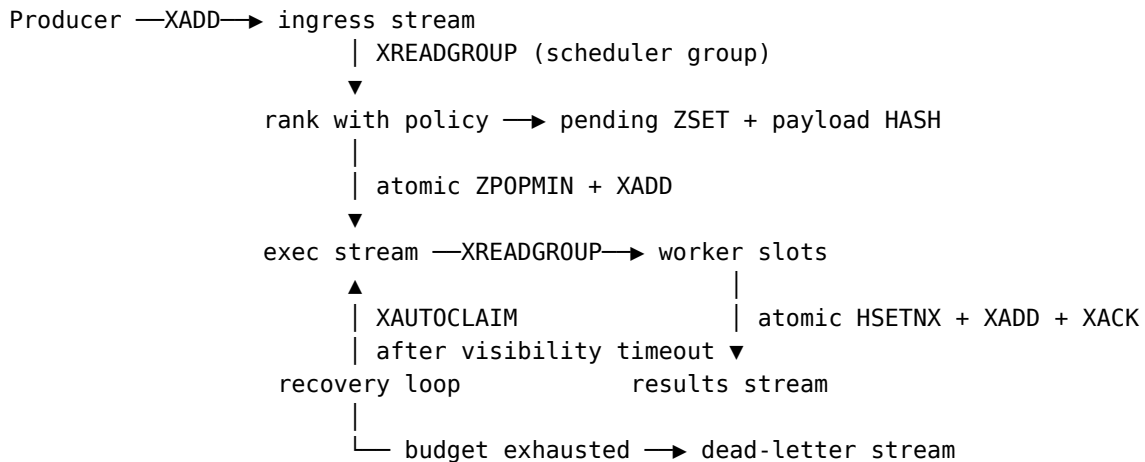
The contribution is empirical rather than algorithmic. All four policies are well established. What is measured is their behaviour under controlled conditions in one concrete system, together with the conditions under which the differences between them matter at all.

1.2 Scope and honest framing

The system is a task queue built on a single Redis instance with a single-writer scheduler. The genuinely hard distributed-systems problems — consensus, partitioning, failure detection, network partitions, clock skew — are absent or delegated to Redis. It is more accurately described as a scheduling study on a Redis-backed queue than as a distributed system, and is described that way throughout.

2. System design

2.1 Data flow



Producers write only to the ingress stream; workers see only what the scheduler has already ranked and dispatched. `max_in_flight` bounds dispatched-but-unfinished work so the backlog remains in the scheduler’s pending set, where the policy controls it, rather than in the execution stream, where it would not.

2.2 The scheduling abstraction

A policy converts a task into an ordering key and nothing else:

```
type Key struct {  
    Score float64 // lower dispatches first  
    Member string // "<zero-padded submit millis>|<task id>"  
}
```

The surrounding machinery always dispatches the smallest key. This exploits a property of Redis: sorted sets order equal-score members lexicographically by byte value, which is exactly Go string comparison. The consequence is that the in-memory priority queue used by unit tests and the ZPOPMIN used in production **cannot disagree**, so the ordering asserted in tests is the ordering the deployed system produces. An integration test verifies the equivalence against a live Redis.

2.3 Policies

FIFO orders by submission time. The experimental baseline.

Strict priority packs priority and submission time into one float64 as $(255 - \text{priority}) \cdot 10^{13} + \text{submitted_millis}$. Unix milliseconds stay below 10^{13} until the year 2286, so a timestamp cannot leak into an adjacent priority band, and the largest score remains inside the range where float64 represents integers exactly. No ageing: starvation is measured, not mitigated.

EDF orders by absolute deadline, with tasks lacking deadlines sorted last.

WFQ implements self-clocked fair queuing over tenants. On admission of task t from tenant k with weight w_k :

```
start = max(virtualTime, lastFinish[k])  
finish = start + cost(t) / w_k
```

and on dispatch, $\text{virtualTime} = \max(\text{virtualTime}, \text{finish})$. Because a newly active tenant starts at $\max(\text{virtualTime}, \text{lastFinish})$, an idle tenant cannot bank credit and flood the queue. Documented approximations: virtual time advances on dispatch events rather than by emulating a fluid server; scheduling is non-preemptive at whole-task granularity; and $\text{cost}(t)$ uses the task's declared duration, an oracle assumption that Section 5.3 shows is consequential.

2.4 Correctness

Four Lua scripts carry the system's correctness, each atomic inside Redis. Dispatch performs ZPOPMIN and XADD in one unit, so a task cannot be lost between selection and delivery. Completion is guarded by HSETNX on a hash of completed task IDs, in the same script that writes the result record and acknowledges the delivery.

The guarantee is stated precisely: **at-least-once delivery with completion recorded exactly once per task ID**. A task body may execute more than once; exactly-once delivery is not claimed. This was verified under a genuine race by setting the visibility timeout below the task duration, so the recovery loop reclaims work that is still legitimately running and two workers race to complete the same task. Duplicate completions were suppressed and every task appeared exactly once in the results stream.

3. Methodology

3.1 Experimental design

Four schedulers $\times$ four workloads, repeated with controlled seeds. Each experiment runs in its own Redis key namespace, and every CSV row repeats the full run identity, so results from different runs cannot be pooled by accident.

Workloads are finite and seeded. For a given seed and configuration the generated task sequence is byte-identical on any machine: every random value comes from one seeded source consumed in a fixed order per task. Timing is not reproducible, only the workload; repetitions and reported variance are the mitigation.

3.2 Offered load

Comparing policies across workloads is only meaningful at equal load relative to what the system can serve:

```
capacity      = worker_slots / mean_execution_seconds
offered_load = mean_arrival_rate / capacity
```

Computing this per workload proved necessary rather than pedantic. The heavy-tailed workload has a mean service time of 51.4 ms against 50.1 ms for the others, so an identical arrival rate is a different load. More seriously, the bursty workload ignores its nominal arrival-rate parameter entirely — its arrivals derive from the burst schedule — and its duration-weighted mean rate was 500/s against 400/s for the other three. **Every bursty measurement in an earlier revision of this work was therefore taken at $1.57\times$ capacity while the others sat at $1.25\times$** , which is a confound, not a rounding difference, and explains why bursty showed a 0.94 deadline miss rate: it was simply run harder.

The load sweep is valid only if changing the arrival rate changes nothing else. It does not: the generator draws arrival gap, tenant, duration, priority and deadline jitter in a fixed order per task, so altering the rate alters gap values without moving the stream position. A test asserts byte-identical durations, priorities, tenants and deadlines across a ninefold rate change for all four workload types.

3.3 Repetition counts

How many repetitions a workload needs is not uniform. All four were re-run at 25 repetitions to find out, giving a 5-versus-25 comparison over 96 workload/scheduler/metric combinations.

workload	5-rep CIs missing the truth	median shift	verdict
uniform	8 / 24	0.60%	converged
bursty	1 / 24	0.13%	converged
multi_tenant	0 / 24	0.26%	converged
heavy_tailed	1 / 24	8.42%	not converged

Interval coverage is the wrong convergence diagnostic, and uniform proves it. uniform is the most stable workload in the matrix and has by far the *most* confidence intervals that missed — eight, against one for the genuinely unconverged heavy-tailed workload. The mechanism is mechanical: a stable metric produces a narrow interval that a 1% shift escapes, while a noisy metric produces a wide interval that almost nothing escapes. Coverage measures how stable a metric is, not how converged the estimate is. The **median shift in the point estimate** separates the workloads cleanly where coverage does not.

The distinguishing property is the task-duration distribution, not the arrival pattern: bursty arrivals and skewed tenants both average out over 4,000 tasks; a Pareto tail does not.

3.4 Environment

macOS 26.6 on an Apple M5 (10 cores), Redis 8.10.1 on localhost, Go 1.27. The main matrix uses 4 worker processes $\times$ 4 slots = 16 slots against a capacity of approximately 319 tasks/s. Real-work experiments use 8 slots on 10 cores, for reasons given in Section 5.3.

4. Results

4.1 Main matrix

Twenty-five repetitions per cell at $1.25\times$ offered load, 400 experiments, 1.6 million tasks. Latency and wait columns are seconds.

workload	sched	p50	p99	maxwait	miss	tasks/s	Jain
uniform	fifo	1.486	2.917	2.900	0.791	308.8	0.999
uniform	prio	0.109	4.491	4.492	0.462	308.3	0.999
uniform	edf	1.483	2.992	3.069	0.788	308.3	0.999
uniform	wfq	1.476	3.240	3.270	0.786	308.2	0.999
bursty	fifo	3.216	6.245	6.273	0.943	308.9	0.999
bursty	prio	1.705	7.538	7.561	0.704	308.1	0.999
bursty	edf	3.158	6.306	6.389	0.937	308.1	0.999
bursty	wfq	3.184	6.345	6.389	0.943	308.2	0.999
heavy-tail	fifo	0.749	1.686	1.613	0.482	282.7	0.979
heavy-tail	prio	0.096	2.685	2.681	0.343	281.7	0.979
heavy-tail	edf	0.031	1.271	6.163	0.001	257.6	0.979
heavy-tail	wfq	0.394	2.487	2.574	0.354	270.5	0.979
multi-ten	fifo	1.479	2.920	2.907	0.780	308.4	0.246
multi-ten	prio	0.105	4.462	4.467	0.457	308.4	0.246
multi-ten	edf	1.459	2.976	3.044	0.775	308.3	0.246
multi-ten	wfq	1.490	2.932	2.910	0.727	308.4	0.246

Throughput is flat within 0.3% across policies on three of four workloads, as it should be: work-conserving schedulers change who waits, not how much work gets done. The exception is heavy-tailed, where EDF delivers 257.6 tasks/s against FIFO’s 282.7 and utilisation falls to 0.73 from 0.80 — deferring long tasks leaves slots idle at the end of a run while the deferred work drains.

4.2 Starvation under strict priority

p95 latency by priority level, uniform workload, 25 repetitions pooled over 100,000 tasks per scheduler:

sched	prio 0	prio 1	prio 2	prio 3	prio 4	spread
fifo	2.796	2.797	2.797	2.797	2.819	1.0×
priority	4.416	0.079	0.075	0.074	0.073	60.4×
edf	2.816	2.813	2.809	2.811	2.834	1.0×
wfq	2.954	2.953	2.947	2.941	2.967	1.0×

Everything above priority 0 completes in about 75 ms while priority 0 waits 4.4 s. FIFO, EDF and WFQ show a flat 1.0×, because none of them reads the priority field.

4.3 Fairness under tenant skew

Jain’s fairness index is defined over completed service seconds per tenant:

$$J(x) = (\sum x_i)^2 / (n \cdot \sum x_i^2)$$

with zero-service tenants included as zeros — omitting them would report perfect fairness for a queue that served exactly one tenant.

The index cannot distinguish the policies on this workload, and it is important to say why. Under the 90/3/3/2/2 skew every policy reports 0.246, near the $1/n = 0.2$ floor, because the index is *demand-limited*: tenants B–E submit a few percent of the work, so no scheduler can grant them a large share of service. That is a property of the workload, not a failure of the scheduler.

Per-tenant latency is the metric that does distinguish them:

sched	A (90%)	small (mean)	ratio
fifo	2.798	2.762	0.987×
priority	4.253	4.191	0.986×
edf	2.791	2.754	0.987×
wfq	2.858	0.078	0.027×

WFQ gives the small tenants 35× lower p95 latency at a 2% cost to the dominant tenant. Under the other three policies the small tenants are queued behind the noisy neighbour’s backlog and see essentially its latency — equal treatment that is not isolation. Reporting the fairness index alone would have shown four identical bars and missed the effect entirely.

4.4 Policy choice depends on load

Seven load levels from 0.5× to 3× capacity, 336 experiments, 1.34 million tasks. Spread between best and worst policy p99 on the uniform workload:

offered load	0.50×	0.75×	1.00×	1.25×	1.50×	2.00×	3.00×
p99 spread	0.1%	0.3%	93.1%	55.5%	28.4%	2.1%	1.1%

Scheduling policy is decisive only in a narrow band around capacity. Below capacity there is no queue, so every policy dispatches immediately. Far above it, the backlog dominates: the makespan is fixed by total work and p99 approaches it whatever order is chosen. This is invisible from any single operating point and is the most generally useful result in the study.

It also carries a practical warning: an evaluation of scheduling policies conducted at an arbitrary load may find no difference and conclude, wrongly, that policy does not matter.

4.5 EDF under overload

An earlier revision of this work asserted in three places that EDF “degrades sharply under overload”, following classical theory, which warns that EDF keeps preferring already-doomed

tasks and misses them anyway while delaying tasks that were still achievable. **The data does not support that here.** Deadline miss rate on the heavy-tailed workload:

offered load	0.50×	1.00×	1.25×	1.50×	2.00×	3.00×
fifo	0.000	0.123	0.645	0.799	0.860	0.893
priority	0.000	0.154	0.415	0.453	0.508	0.683
edf	0.000	0.000	0.005	0.368	0.635	0.769
wfq	0.000	0.088	0.507	0.690	0.798	0.864

EDF has the lowest miss rate at every load tested; its advantage narrows from roughly 129× better than FIFO at 1.25× to 1.16× at 3×, but never inverts. The classical pathology requires deadlines that are tight relative to service time, whereas these are generous and scale with each task’s own duration, so a deferred long task usually still meets its deadline. The result is therefore a property of the workload as much as of the policy.

What EDF does pay, at every load, is starvation: its longest queue wait is three to four times FIFO’s and saturates near 9.6 s, because it defers precisely the large tasks whose deadlines are furthest away. Miss rate and maximum wait must be read as a pair.

4.6 The cost of centralisation

The scheduler is a deliberate single-writer component. Two measurements establish what it costs.

Its own capacity. With workers removed and the critical path driven directly:

stage	rate	note
dispatch, best case	271,198 tasks/s	batch 256; atomic ZPOPMIN+XADD
admission	15,299 tasks/s	one Redis round trip per task
end to end	11,562 tasks/s	both loops — the real ceiling

Dispatch is 18× faster than admission, so dispatch was never the constraint: it amortises a batch over one round trip while admission pays one per task. The scheduler can feed approximately **578 worker slots** at a 50 ms mean task. It follows that every experiment in this study ran the scheduler at about **3% of its capacity**, so the policy comparisons measure policy rather than scheduler saturation — a validity check the work could not previously make.

Its overhead, against no scheduler at all. A control arm was added in which workers consume the ingress stream directly, producing the same execution order as FIFO. 240 experiments, 960,000 tasks.

offered load	uniform	bursty	heavy_tailed	multi_tenant
0.75×	0.0%	0.2%	0.0%	0.0%
1.25×	1.5%	1.6%	1.2%	1.6%

Centralisation is unmeasurable below capacity and costs about 1.5% above it: the extra round trip through the pending set. What that enables costs considerably more and returns considerably more (heavy-tailed, 1.25×):

arm	miss	maxwait	tasks/s	vs. none
none	0.609	2.09	272.7	—
fifo	0.644	2.29	269.4	−1.2%
priority	0.415	3.69	262.3	−3.8%
wfq	0.508	3.38	255.2	−6.4%
edf	0.005	8.25	244.0	−10.5%

EDF trades 10.5% throughput for a 122× reduction in deadline misses; WFQ trades 6.4% for tenant isolation. **FIFO is the one arm that fails to justify itself**: it costs 1.2% and delivers identical ordering, so it is strictly worse than having no scheduler. FIFO earns its place here as an experimental baseline, not as a deployment choice.

5. Threats to validity

5.1 Corrections made during this study

Three quantitative claims were found to be wrong after more data was collected.

The bursty load confound (Section 3.2): bursty experiments ran at 1.57× capacity while others sat at 1.25×, because the workload ignores its nominal arrival-rate parameter. Corrected by deriving arrival rate per workload from measured capacity.

EDF’s overload behaviour (Section 4.5): asserted from theory in three places before measurement, and contradicted by the sweep.

EDF’s real-work degradation (Section 5.3): first reported as 27× from three repetitions on the one workload already shown to require twenty-five. At n=25 the correct figure is 8.4×. The direction held; the magnitude was inflated more than threefold, and the sleep-mode value it was measured against was a lucky draw — 0.099 ± 0.183 at n=25, a standard deviation larger than its own mean.

5.2 Simulated execution

Workers sleep for a task’s declared duration by default. This makes task sizes exactly controllable and removes application variance, at the cost that a sleeping worker consumes no CPU: N slots always deliver N-way parallelism regardless of core count, and utilisation measures slot occupancy rather than work.

5.3 The sleep model is not neutral

To test whether the conclusions survive real work, an execution mode was added that burns each task’s duration in chained SHA-256 — chosen because it is data-dependent and cannot be strength-reduced away — with iterations calibrated per machine. The same experiment was run both ways at 8 slots on 10 cores, to keep capacity slot-bound rather than core-bound.

Measured execution inflates by 1.5–1.9× even at 8 slots on 10 cores, because worker goroutines share the machine with Redis and the Go runtime. Deadline miss rate on the heavy-tailed workload, 25 repetitions:

sched	sleep	cpu	degrade	inflation	p99 ratio
edf	0.099 ± 0.183	0.829 ± 0.173	8.4×	1.88×	4.92×
priority	0.408 ± 0.046	0.643 ± 0.078	1.6×	1.85×	3.19×
wfq	0.474 ± 0.147	0.851 ± 0.039	1.8×	1.69×	3.07×
fifo	0.675 ± 0.130	0.866 ± 0.048	1.3×	1.54×	3.41×

Under real CPU, **strict priority overtakes EDF** (0.643 against 0.829, separable at n=25) and EDF’s advantage over FIFO nearly vanishes. EDF shows both the largest execution inflation and the worst p99 degradation of any policy.

WFQ, by contrast, is unaffected. Per-tenant p95 latency under skew:

exec mode	small p95, fifo	wfq	advantage
sleep	2.750 s	0.086 s	31.8×
cpu	5.299 s	0.133 s	39.9×

The opposite was predicted, on the grounds that WFQ computes virtual finish times from declared cost exactly as EDF computes ordering from declared deadlines. The reason it does not degrade is the more interesting finding:

WFQ’s use of cost is relative; EDF’s is absolute. WFQ compares tenants against one another, so a cost error inflating every task by roughly the same factor is common-mode and cancels in the ratio. EDF compares each task against a wall-clock deadline, where the same error does not cancel. A cost model can be badly wrong and still produce correct fair-queuing decisions; it cannot produce correct deadline decisions.

FIFO degrades least of all, for an instructive reason: it reads no task metadata whatsoever, so nothing about it depends on the duration model being accurate. Under a wrong cost model, knowing nothing is a form of robustness.

The practical consequence is that the EDF results in Sections 4.1 and 4.5 should be read as an upper bound attainable only with perfect cost information. The WFQ fairness results are not subject to that caveat: they were re-tested under real work and improved.

5.4 Remaining limitations

1. **Single-node Redis.** No cluster, no failover. Redis failure modes are out of scope, and the Lua scripts, while declaring all their keys, have not been validated against Redis Cluster key-slot constraints.
2. **Single-writer scheduler.** Scheduling throughput is bounded by one process, which bounds the scale at which these results apply.
3. **At-least-once only.** Duplicate execution is expected; only completion recording is idempotent.
4. **Fault injection is shallow.** Worker crashes and application errors only — no Redis failover, network partition, or gray failure.
5. **No external baseline.** No comparison against Celery, Sidekiq or Temporal, so absolute throughput figures have no external reference point.
6. **Timing is not reproducible,** only the workload. Cross-machine comparison is not supported.
7. **The real-work comparison is narrow:** three of four workloads, a single offered load, one machine's core count.

6. Conclusions

Scheduling policy matters in a narrow band. The spread between best and worst policy moves from 0.1% below capacity to 93% at capacity and back to 1.1% at 3× capacity. Evaluations conducted at a single arbitrary load risk concluding that policy is irrelevant.

Centralised scheduling is cheap; using it is not, and it is worth it. Centralisation costs about 1.5% of throughput. Acting on it costs 6–10% and returns a 122× reduction in deadline misses (EDF) or 35× lower latency for small tenants (WFQ). FIFO alone does not repay its own overhead.

Metric choice can hide the effect entirely. Jain's index over completed service reports 0.246 for every policy under tenant skew, because it is demand-limited. The 35× isolation WFQ provides is visible only in per-tenant latency.

The sleep model systematically flatters deadline scheduling. EDF's advantage is contingent on cost information that real execution invalidates; WFQ's is not, because its cost errors are common-mode. Any study of deadline-aware scheduling that simulates execution with sleeps should be read with this in mind.

Diagnostics can invert. Confidence-interval coverage, the obvious test for whether enough repetitions have been run, ranked the most-converged workload worst and the least-converged workload well. The magnitude of shift in the point estimate is the diagnostic that works.

6.1 Future work

Cost *estimation* from observed durations rather than declared ones is the natural next step, and Section 5.3 predicts it would recover much of EDF's lost advantage while leaving WFQ unchanged. Beyond that: ageing or lottery scheduling as a starvation-free alternative to strict priority; WF2Q or deficit round robin to quantify what self-clocking costs; a tenant-sharded scheduler to lift the single-writer bound; Redis failover and partition injection; and latency-aware autoscaling driven by queue depth rather than CPU.

Appendix A: Reproducing these results

```
make redis-up
make bench-full REPETITIONS=25 RUN=main          # Section 4.1
make sweep RUN=sweep-loads REPETITIONS=3         # Section 4.4
make ceiling TASKS=20000                          # Section 4.6
go run ./cmd/benchmark --schedulers none,fifo,priority,edf,wfq \
  --loads 0.75,1.0,1.25,1.5 --repetitions 3 --run-id control-arm
go run ./cmd/benchmark --config experiments/realwork.yaml \
  --exec-mode cpu --repetitions 25 --run-id realwork # Section 5.3
make plots RUN=main && make sweep-plots RUN=sweep-loads
```

Source, configurations and full result schemas: <https://github.com/anishc23/distributed-task-queue-scheduler>

Appendix B: Experiment inventory

run	experiments	tasks
Main matrix, 25 repetitions	400	1,600,000
Offered-load sweep	336	1,344,000
No-scheduler control	240	960,000
Real work, heavy-tailed, 25 reps	200	400,000
Real work, initial comparison	48	96,000
Real work, tenant isolation	40	80,000
Total	1,264	4,480,000